Inverted Indexes

map every word to the documents that contain it

Watch the 4-min video

In one line: an inverted index maps each **term** to a sorted **postings list** of the documents (and positions) where it appears, so a query becomes a few dictionary lookups plus list intersections and unions; the index is compressed and written as **immutable segments** that are merged in the background.

1 Never read documents at query time

You have 10 million documents and someone searches **distributed cache**. Scanning every document's text, even at a few GB/s, takes seconds to minutes. Search engines answer in milliseconds because the expensive work happened at index time: they read a precomputed **inverted index**, which is the same idea as the index at the back of a book.

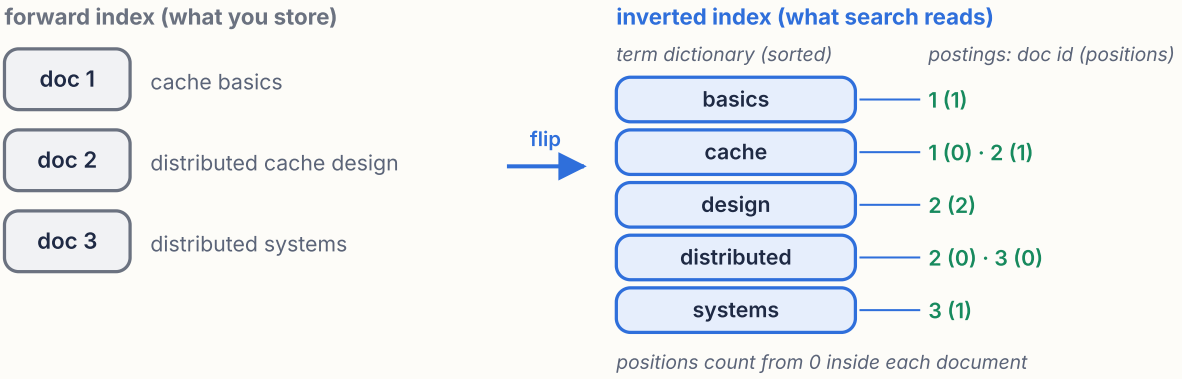


Figure 1. The three documents from the video. A forward index maps doc → words; the inverted index maps term → postings.

The index has two parts. The **term dictionary** holds every distinct term, sorted, so lookups and prefix scans are fast (Lucene stores it as blocks on disk with a small in-memory index built as an FST, a compact automaton over the term bytes). Each term points to its **postings**: the doc IDs that contain it, usually with the **term frequency** (for scoring, lesson 17) and the **positions** (for phrases). Doc IDs are small integers assigned per segment, not your database IDs.

2 Queries are list operations

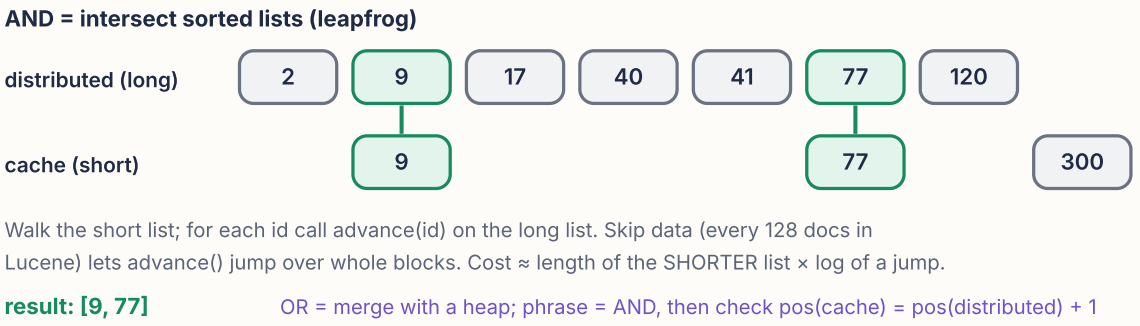


Figure 2. Intersection walks the shorter list and jumps ahead in the longer one, so rare terms make AND queries cheap.

- **AND**: intersect the postings lists. Start with the rarest term; because lists are sorted by doc ID, the engine leapfrogs: ask the next list to `advance()` to the current candidate, using skip data to jump over blocks.
- **OR**: union with a priority queue. For top-k results, Lucene's **block-max WAND** skips blocks whose best possible score can't reach the current top k. This is why Elasticsearch counts hits exactly only up to 10,000 by default (`track_total_hits`): an exact count would force it to visit every match.
- **NOT**: subtract one list from another.
- **Phrase** 'distributed cache': intersect first, then check positions: cache must be at position p + 1 where distributed is at p. **Proximity** ('within 5 words', `slop`) relaxes that check.
- **Prefix / wildcard / fuzzy**: first expand the pattern against the sorted term dictionary into a set of terms, then OR their postings. A pattern that expands to thousands of terms is slow, which is why leading wildcards are discouraged even here.

3 Compression: why indexes fit in memory

Postings are stored as **gaps** between consecutive doc IDs instead of the IDs themselves. Sorted IDs make the gaps small, and small numbers need few bits: after the first, these gaps fit in 6 bits each instead of 32.

```
IDs 1000, 1003, 1010, 1011, 1050 → gaps 1000, 3, 7, 1, 39
```

Lucene packs gaps in blocks of 128 using the bit width of the largest gap in the block (frame-of-reference bit-packing), with variable-byte encoding for the tail. Common terms have dense lists and tiny gaps, so they compress best. Compressed lists mean fewer bytes from disk and more of the index in the OS page cache; decoding a packed block is very fast on modern CPUs, often faster than reading the uncompressed data.

4 Updates: immutable segments plus merging

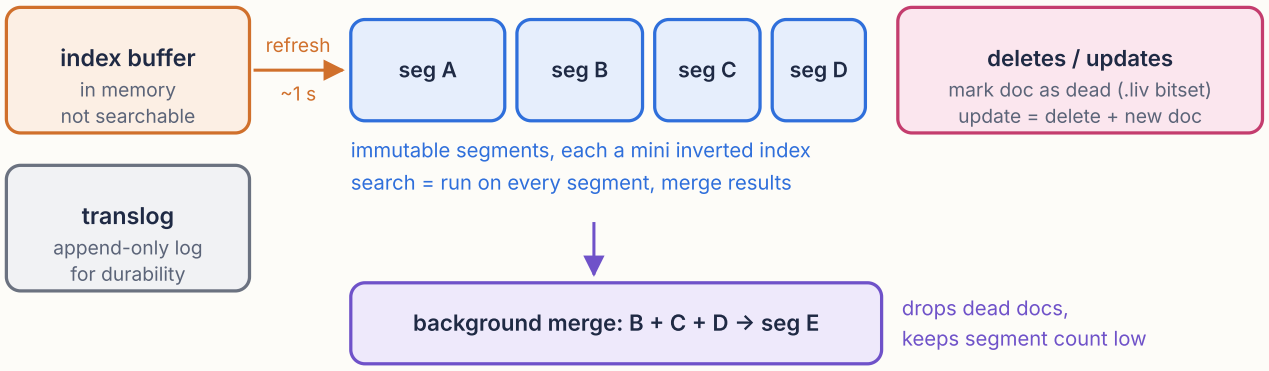


Figure 3. Lucene's write path. New documents become new segments on refresh; deletes are marks; merges rewrite segments in the background.

Rewriting a sorted, compressed postings list for every new document would be very slow, so Lucene never edits a segment in place. New documents collect in an in-memory buffer; a **refresh** (every 1 s by default in Elasticsearch, lesson 20) turns the buffer into a new small, searchable segment. A **delete** just sets a bit in the segment's live-docs bitset; an **update** is a delete plus a new document. Durability comes from a separate **translog** that is replayed after a crash; a **flush** does a full Lucene commit (fsync) and trims the translog.

Many small segments slow every query, because each query visits each segment. A **merge policy** (Lucene's TieredMergePolicy by default) merges similar-sized segments into bigger ones, physically removing deleted documents. Merges cost disk I/O and temporary disk space, which is why heavily-updated indexes need headroom, and why 'force merge to one segment' is only for indexes that will never be written again (e.g. yesterday's logs).

WORKED EXAMPLE: THREE DOCUMENTS

Doc 1 'cache basics', doc 2 'distributed cache design', doc 3 'distributed systems'. After analysis the dictionary is basics, cache, design, distributed, systems. `cache → [1, 2]`, `distributed → [2, 3]`. The query **distributed AND cache** intersects [2, 3] with [1, 2]: the leapfrog starts at 2 in both lists → match; next 3 vs end → done. Result: **doc 2**. For the phrase "distributed cache", doc 2 has distributed at position 0 and cache at position 1, so it also matches as a phrase. No document text was read. Now delete doc 2: its bit in the live-docs set flips to 0, the postings stay until the next merge, and every query simply skips it.

Stored per term or doc	Used for	Cost if you don't need it
Doc IDs	matching	always needed
Term frequencies	BM25 scoring	small; disable only for pure filter fields
Positions	phrase and proximity queries	often the biggest part of the index; keyword fields skip them
Offsets	fast highlighting	extra space; off by default
Doc values (columnar)	sorting, facets (lesson 18)	a separate structure, not part of the inverted index

TRADE-OFFS

- **Read speed vs write cost:** everything is precomputed, so writes do analysis, buffering and later merging.
- **Positions** enable phrases but can double the index size.
- **Immutability** makes caching and concurrency simple but means deletes cost space until merged.
- **Frequent refresh** means fresh results but many tiny segments and more merging.

COMMON MISTAKES

- Thinking an update modifies a document in place: it rewrites the whole document and leaves a deleted copy behind.
- Updating one hot document (a counter) thousands of times a second: each update creates a deleted doc and churns segments.
- Force-merging an index that is still being written to: it creates huge segments that the normal merge policy then avoids.
- Treating hit counts as exact by default: past 10,000 Elasticsearch reports 'gte 10000' unless you ask for exact totals.

5 Key terms

Term dictionary: sorted set of all terms, points to postings

Skip data: pointers that let intersection jump ahead in a list

Segment: immutable mini-index; a shard is many segments

Postings list: sorted doc IDs (+ freq, positions) for one term

Delta / gap encoding: store differences between sorted IDs

Merge: combine segments, purge deleted docs

6 Interview questions

Q1. How does an inverted index work? BASIC

It maps every term to a sorted postings list of the documents containing it, with positions for phrases; a query looks up each term and intersects or unions the lists.

Q2. How do engines keep indexes fast to update? BASIC

They write new immutable segments, mark deletions in a bitset instead of rewriting, and merge segments in the background.

Q3. How is a phrase query answered? BASIC

Intersect the terms' postings, then use stored positions to keep only documents where the terms appear at consecutive positions.

Q4. An AND query with one very common and one very rare term: which list drives? SCENARIO

The rare term's short list: iterate it and advance into the long list with skip data, so the cost tracks the short list.

Q5. Index size doubled after enabling a feature on a text field. What likely happened? SCENARIO

Positions or offsets (or n-grams) were enabled on a large field; check whether you really need phrase queries or fast highlighting on it.

Q6. Why are deletes 'cheap' but not free? TRICKY

A delete only flips a bit, but the document still occupies space and is still visited and skipped by queries until a merge rewrites the segment.

Q7. Why does compression make search faster, not slower? TRICKY

Gap-encoded, bit-packed postings are far smaller, so more fits in RAM and CPU caches, and decoding packed blocks is cheaper than reading extra bytes from memory or disk.

Q8. Why is a wildcard like *cache slow even in a search engine? TRICKY

The term dictionary is sorted by prefix; a leading wildcard must scan every term to find matches and then OR many postings lists. Index a reversed or n-gram sub-field instead.

Next: SD-P-003 · Elasticsearch Architecture backendroadmap.com